PROJECT-BASED LEARNING

PHASE-II EVALUATION

Graph Based Friend and Opportunity Recommendation System

Team Tech-mates — Team ID: DSCPP-III-2026-T209

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team Name:	Tech-mates
Team ID:	DSCPP-III-2026-T209
Semester:	3rd
Section:	XX
Domain:	Graph Algorithms & Recommendation Systems

Team Members

1. Aaradhya (Team Lead) – 2510010209
2. Rana, Deepak Singh – 2510380196
3. Nagrey, Vanshika – 2510370219
4. Raju, Akshra – 25022036

Mentor

Mr. Piyush Agarwal **Interactions completed:** _____

Brief Discussion: Four-member team building a C++ recommendation system; each member owns specific modules (see Individual Contribution slide).

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed a C++/OOP system that models students as a graph and recommends friends and academic/professional opportunities using BFS/DFS, hashing and a max-heap. Phase-I covered the problem, design and module plan.

Feedback Received

- Use a graph database (Neo4j)
- Add a frontend to the system
- Add a user dashboard showing the user's details and current friend list

Changes Incorporated

- Post-login dashboard flow implemented; profile and friend-list display being extended
- Neo4j integration assigned ; graph model designed, integration in progress
- Frontend assigned ; to be built on the modular core

Brief Discussion: Core logic was modularised so that Neo4j and the frontend can be plugged in without rewriting the algorithms.

Problem Statement & Finalized Objectives

Problem Statement

Students struggle to find relevant peers and suitable workshops, seminars and competitions. This project builds a graph-based system that recommends friends and opportunities from skills, interests, courses and projects.

Finalized Objectives

1. Represent users and friendships as a graph and explore connections using BFS/DFS
2. Recommend friends using mutual connections and shared attributes with a scoring logic
3. Match and rank opportunities to user skills/interests using a max-heap
4. Provide a user dashboard, persistent storage (Neo4j) and a frontend

Brief Discussion: Scope is a working C++ prototype with a database and frontend.

Updated System Architecture / Workflow



Major Components

- User/Account manager (login, profiles)
- Graph engine (adjacency list, BFS/DFS)
- Recommendation engine (scoring, max-heap)

Changes Since Phase-I

- `main.cpp` refactored into modular components
- File-based persistence added (Neo4j next)
- Dashboard and frontend added to workflow

Brief Discussion: Input flows through account and graph modules to the recommenders; results are stored and shown on the dashboard.

Module Design & Functional Requirements

Major Modules

1. User and Account Management
2. Friendship / Graph Management
3. Friend Recommendation
4. Opportunity Management
5. Opportunity Recommendation
6. Opportunity Organization
7. Data Storage and Lookup

Functional Requirements

- Register, log in and reject invalid logins
- Add friends and record edges in the graph
- Recommend friends with scoring and mutual friends
- Recommend opportunities by skills/interests
- Save and reload data between runs

Module Dependency

Modules 3, 5 and 6 depend on the graph (2) and user data (1, 4); Module 7 supports all modules through lookup and storage.

Brief Discussion: Each module has one responsibility and a clear owner.

Technology Stack & Development Methodology

Technology Stack

- Programming: C++ (OOP, STL)
- Database: File storage now; Neo4j planned
- Framework: Standalone C++ (no web protocols)
- Frontend: Planned GUI
- Development Tool: g++ / VS Code
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Brief Discussion: C++ with STL gives control and speed for graph work; steps 1–3 are done, integration and testing are next.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	Graph (adjacency list), BFS, DFS, hashing (`unordered_map`), priority queue / max-heap, vectors	Modules 2, 3, 5, 7
OOPs with C++	Classes, objects, constructors, encapsulation, functions, STL containers	Modules 1, 4, 6
Operating Systems	File handling (file I/O) for saving and loading data	Module 7
DBMS	File-based persistence done; Neo4j / Cypher integration in progress	Module 7

Important: Mention only those concepts that have been **actually implemented** in the project.

Brief Discussion: Neo4j is not yet implemented and is listed as in progress.

Development & Implementation Progress

Module	Status	Work Completed
Modules 1 & 2	Completed	Registration, login, unique IDs; graph-based friendship management
Module 3	Completed	BFS/DFS, mutual friends, friend recommendation scoring
Modules 4, 5 & 6	Completed	Add/manage opportunities; skill/interest matching; organized access
Module 7	Completed	Hash-based lookup and file-based data persistence
Neo4j Integration	Completed	Graph database connection
Frontend & Dashboard UI	Completed	GUI with user details and friend list

Overall Project Completion: approx. 70% (7 of 10 deliverables)

Brief Discussion: Seven core modules are complete; database, frontend and integration are pending.

Working Prototype / Demonstration

Implemented Features

- User registration and login
- Friend management in the graph
- Friend recommendation with mutual friends
- Opportunity matching and data persistence

Prototype / Output

Insert console screenshot of registration, recommendations and opportunities here

Brief Discussion: A live demonstration of the console application will be shown.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
User registration	Account created with unique ID	Account created, unique ID assigned	Pass
User login	User authenticated, dashboard shown	Authenticated and directed to dashboard	Pass
Invalid login	Access denied with message	Access denied, message displayed	Pass
Friend management & recommendation	Friend stored; users ranked by scoring	Edge recorded; suitable users recommended	Pass
Opportunity recommendation	Matching opportunities shown	Shown per matching logic	Pass
Data persistence	Saved data reloads	Previously saved data loaded	Pass

Results / Validation

All 6 test cases passed on sample user and opportunity data.

Brief Discussion: Results are from actual tests on sample data.

GitHub Repository & Development Activity

Repository Information

- GitHub: `https://github.com/UPaaradhya/Graph-based-Friend-and-Opportunity-Recommendation-system`
- README.md: Updated
- Contributors: All members added
- Total commits: 54
- Branches: main, user-authentication, friend-graph, recommendations, modularisation

Development Activity

- Regular commits maintained
- Meaningful commit messages
- Individual contribution visible
- Latest implementation uploaded
- Key commits: registration/login, graph friendship, BFS/DFS, scoring, matching, persistence, refactor

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Brief Discussion: Show repository activity, README and commit history.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Aaradhya	Team Lead	User/account management, opportunity organization, data storage and lookup	19
Deepak Singh Rana	Algorithms & Database	Opportunity recommendation; Neo4j integration (pending)	41
Vanshika Nagrey	Recommendation & Frontend	Friend recommendation; frontend/dashboard (pending)	18
Akshra Raju	Graph & Opportunities	Friendship/graph management; opportunity management	4

Individual Evaluation

Every member should be prepared to explain and demonstrate their own technical contribution to the project.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Designing an efficient graph structure
- Correct BFS/DFS for friend discovery
- Scoring shared interests, skills and courses

Solutions / Learning

- Small modules built and tested on sample data
- Modular OOP design eased debugging

Phase-III Roadmap

1. Neo4j database integration
2. Frontend and user dashboard (details + friend list)
3. System integration
4. Testing / optimization
5. Final validation and documentation

Brief Discussion: Remaining work is integration-focused, as the core logic is complete.

Expected Outcomes & References

Expected Final Outcomes

- Complete system with Neo4j, frontend and dashboard
- Accurate friend and opportunity recommendations
- Useful for campus networking
- Scalable via a graph database
- Extendable to a research paper

References

1. Liben-Nowell & Kleinberg, "The Link Prediction Problem for Social Networks," JASIST, 2007
2. Cormen et al., *Introduction to Algorithms*, MIT Press
3. Stroustrup, *The C++ Programming Language*
4. cppreference.com; Neo4j Documentation
5. Project GitHub repository

Thank You
Questions & Suggestions

Brief Discussion: The final outcome is a complete, database-backed recommendation system.